

PowerShell — Syntaxe et instructions courantes

Support de référence — langage, pipeline, objets et cmdlets utiles en administration Microsoft 365.

Complément du [module 7 — Outils d'administration et PowerShell](#).

Table des matières

1. [Premiers pas](#)
 2. [Les cmdlets](#)
 3. [Variables et types](#)
 4. [Chaînes de caractères](#)
 5. [Tableaux et hashtables](#)
 6. [Le pipeline](#)
 7. [Opérateurs](#)
 8. [Structures de contrôle](#)
 9. [Fonctions et scripts](#)
 10. [Modules et connexion M365](#)
 11. [Gestion des erreurs](#)
 12. [Récapitulatif des instructions courantes](#)
-

1. Premiers pas

1.1 Qu'est-ce que PowerShell ?

PowerShell est un **shell** (interpréteur en ligne de commande) et un **langage de script** orienté objets. Chaque commande retourne des objets, pas seulement du texte.

ÉLÉMENT	DESCRIPTION
Console	<code>pwsh</code> (PowerShell 7+, multiplateforme) ou <code>powershell</code> (Windows PowerShell 5.1)
Cmdlet	Commande native PowerShell : <code>Verbe-Nom</code> (ex. <code>Get-Process</code>)
Module	Bibliothèque de cmdlets (<code>Microsoft.Graph</code> , <code>MicrosoftTeams</code> ...)
Script	Fichier <code>.ps1</code> exécutable

1.2 Commandes de base

```
# Afficher la version
$PSVersionTable.PSVersion

# Lister les commandes disponibles
Get-Command

# Aide sur une cmdlet
Get-Help Get-Process
Get-Help Get-Process -Examples
Get-Help Get-Process -Full

# Mettre à jour l'aide locale (une fois)
Update-Help
```

1.3 Exécution de scripts

```
# Politique d'exécution (souvent Restricted par défaut sur Windows)
Get-ExecutionPolicy
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

# Lancer un script
.\mon-script.ps1
.\mon-script.ps1 -Parametre "valeur"
```

POLITIQUE	EFFET
Restricted	Aucun script
RemoteSigned	Scripts locaux OK ; scripts téléchargés doivent être signés
Bypass	Tout autorisé (session temporaire, tests uniquement)

2. Les cmdlets

2.1 Convention Verbe-Nom

Toute cmdlet suit le schéma `Verbe-Nom` :

VERBE	ACTION	EXEMPLE
<code>Get</code>	Lire	<code>Get-MgUser</code>
<code>Set</code>	Modifier	<code>Set-MgUser</code>
<code>New</code>	Créer	<code>New-MgUser</code>
<code>Remove</code>	Supprimer	<code>Remove-MgUser</code>
<code>Connect</code>	Se connecter	<code>Connect-MgGraph</code>
<code>Import</code> / <code>Export</code>	Fichiers	<code>Import-Csv</code> , <code>Export-Csv</code>

```
# Syntaxe générale
Get-Process -Name "explorer"
Get-Process | Where-Object CPU -gt 100

# Paramètres positionnels (ordre important)
Get-ChildItem C:\Temp -Recurse -File
```

2.2 Alias (raccourcis)

```
Get-Alias # liste des alias
Get-Alias -Name ls, dir, cd # alias courants

# Équivalences fréquentes
ls # → Get-ChildItem
dir # → Get-ChildItem
cd # → Set-Location
cat, type # → Get-Content
echo # → Write-Output
```

À retenir — En script, préférez toujours le nom complet de la cmdlet (`Get-ChildItem`) plutôt que (`ls`) pour la lisibilité.

3. Variables et types

3.1 Déclaration

```
# Préfixe $ obligatoire à l'utilisation
$nom = "Alice"
$age = 30
$satif = $true

# Types courants (inférence automatique)
[int]$compteur = 42
[string]$texte = "hello"
[bool]$ok = $false
[datetime]$maintenant = Get-Date
```

TYPE POWERSHELL	EXEMPLE	NOTES
[string]	"texte"	Chaîne
[int]	42	Entier 32 bits
[long]	1L	Entier 64 bits
[double]	3.14	Flottant
[decimal]	19.99	Précision fixe
[bool]	\$true , \$false	Booléen
[array]	@(1, 2, 3)	Tableau
[hashtable]	@{ cle = "val" }	Dictionnaire
[pscustomobject]	[pscustomobject]@{ A=1 }	Objet anonyme

3.2 Variables automatiques utiles

```
$_          # objet courant dans le pipeline
$PSItem     # synonyme de $_
$?          # $true si la dernière commande a réussi
$LASTEXITCODE # code retour du dernier programme externe
$error[0]    # dernière erreur
$HOME       # répertoire utilisateur
$PWD        # répertoire courant
```

3.3 Constantes et portée

```
# Constante (lecture seule après assignation)
Set-Variable -Name Pi -Value 3.14 -Option ReadOnly

# Portée
$global:config = "visible partout"
$script:compteur = 0      # dans le script courant
$local:temp = "fonction"  # dans la fonction courante
```

4. Chaînes de caractères

```
# Guillemets simples = littéral (pas d'interpolation)
$msg = 'Bonjour $nom'

# Guillemets doubles = interpolation et échappement
$msg = "Bonjour $nom, tu as $($age) ans."

# Here-string (multiligne)
$bloc = @"
Ligne 1
Variable : $nom
"@

# Format opérateur
$msg = "Licences : {0} / {1}" -f $consommees, $total

# Concaténation
$schemin = $dossier + "\" + $fichier
$schemin = Join-Path $dossier $fichier  # préféré
```

5. Tableaux et hashtables

5.1 Tableaux

```
# Création
$nombres = 1, 2, 3, 4
$couleurs = @("rouge", "vert", "bleu")
$vide = @()

# Accès (index à partir de 0)
$couleurs[0]      # premier élément
$couleurs[-1]     # dernier élément
$couleurs.Count   # nombre d'éléments

# Ajout (nouveau tableau créé)
$couleurs += "jaune"

# Parcours
foreach ($c in $couleurs) {
    Write-Host $c
}
```

5.2 Hashtable (dictionnaire)

```
# Création
$user = @{
    DisplayName      = "Jean Dupont"
    UserPrincipalName = "jean.dupont@contoso.com"
    UsageLocation    = "FR"
}

# Accès
$user.DisplayName
$user["UsageLocation"]

# Hashtable imbriquée (souvent utilisée avec Graph)
$body = @{
    DisplayName = "Marie Martin"
    PasswordProfile = @{
        Password = "Temporaire2026!"
        ForceChangePasswordNextSignIn = $true
    }
}
```

5.3 Objets personnalisés

```
# PSCustomObject (sortie tabulaire propre)
$resultat = [pscustomobject]@{
    Nom      = "Teams Pro"
    SKU      = "Microsoft_Teams_Premium"
    Unites   = 150
}

# Tableau d'objets → export CSV
$data | Export-Csv -Path ".\rapport.csv" -NoTypeInformation -Encoding UTF8
```

6. Le pipeline

Le pipeline (`|`) envoie la **sortie objet** d'une cmdlet à la suivante.

```
# Chaînage classique
Get-Process |
    Where-Object CPU -gt 50 |
    Sort-Object CPU -Descending |
    Select-Object -First 5 Name, CPU

# Syntaxe alternative (filtre sur propriété)
Get-Process | Where-Object { $_.CPU -gt 50 }

# ForEach-Object (traitement un par un)
1..5 | ForEach-Object { $_ * 2 }

# Abréviations courantes
Where-Object → ?    # filtre
ForEach-Object → %   # boucle pipeline
Select-Object → select
Sort-Object   → sort
```

6.1 Cmdlets de manipulation d'objets

```
# Sélectionner des propriétés
Get-MgUser | Select-Object DisplayName, UserPrincipalName, AccountEnabled

# Propriétés calculées
Get-MgSubscribedSku | Select-Object SkuPartNumber,
    @{ N = "Disponibles"; E = { $_.PrepaidUnits.Enabled - $_.ConsumedUnits } }

# Grouper
Get-MgUser | Group-Object -Property UsageLocation

# Compter
(Get-MgUser -All).Count
Get-MgUser | Measure-Object

# Trier
Get-MgUser | Sort-Object DisplayName

# Dédupliquer
$liste | Select-Object -Unique
```

6.2 Export et import

```
# CSV
Import-Csv -Path ".\utilisateurs.csv"
Get-MgUser | Select DisplayName, UserPrincipalName | Export-Csv .\users.csv -
NoTypeInformation

# JSON
Get-Content .\config.json | ConvertFrom-Json
$objj | ConvertTo-Json -Depth 5 | Set-Content .\config.json

# XML
Export-Clixml -Path .\donnees.xml
Import-Clixml -Path .\donnees.xml
```

7. Opérateurs

7.1 Comparaison

```
-eq      # égal
-ne      # différent
-gt      # supérieur
-ge      # supérieur ou égal
-lt      # inférieur
-le      # inférieur ou égal
-like    # motif avec wildcards (* ?)
-notlike
-match   # expression régulière
-contains / -notcontains  # tableau contient une valeur
-in / -notin                # valeur dans un tableau
```

```
"admin" -like "adm*"
$email -match '@contoso\.com$'
3 -in 1, 2, 3, 4          # $true
```

7.2 Logiques et arithmétiques

```
# Logiques
-and    # &&
-or     # ||
-not    # !
-xor

# Arithmétiques
+ - * / %

# Assignment
+= -= *= /=
```

7.3 Opérateurs spécifiques PowerShell

```
# Appel de méthode sur chaque objet du pipeline
$liste | ForEach-Object { $_.ToUpper() }

# Sous-expression $( )
"Total : $(2 + 3)"

# Cast de type
[int]"42"
[string]123

# Opérateur de plage
1..10           # tableau 1 à 10
foreach ($i in 1..5) { }

# Opérateur ternaire (PowerShell 7+)
$statut = if ($age -ge 18) { "majeur" } else { "mineur" }

# Null-coalescing (PowerShell 7+)
$valeur = $config ?? "défaut"
```

8. Structures de contrôle

8.1 Conditions

```
if ($licence -eq "E5") {  
    Write-Host "Licence premium"  
}  
elseif ($licence -eq "E3") {  
    Write-Host "Licence standard"  
}  
else {  
    Write-Host "Autre SKU"  
}  
  
# Switch (valeurs multiples)  
switch ($role) {  
    "GlobalAdmin" { "Accès total" }  
    "UserAdmin"   { "Gestion utilisateurs" }  
    default       { "Rôle limité" }  
}  
  
# Switch sur propriété d'objet  
switch ($user.AccountEnabled) {  
    $true { "Actif" }  
    $false { "Désactivé" }  
}
```

8.2 Boucles

```
# foreach (collection)
foreach ($user in $utilisateurs) {
    Write-Host $user.DisplayName
}

# foreach (pipeline)
Get-MgUser | ForEach-Object {
    Write-Host $_.UserPrincipalName
}

# for
for ($i = 0; $i -lt 5; $i++) {
    Write-Host $i
}

# while / do until
$compteur = 0
while ($compteur -lt 3) {
    $compteur++
}

do {
    $compteur--
} until ($compteur -eq 0)
```

8.3 Contrôle de flux

```
foreach ($item in $liste) {
    if ($item -eq "skip") { continue } # itération suivante
    if ($item -eq "stop") { break }    # sortie de boucle
    Write-Host $item
}
```

9. Fonctions et scripts

9.1 Fonction simple

```
function Get-Salutation {  
    param(  
        [Parameter(Mandatory)]  
        [string]$Nom,  
  
        [int]$Age = 0  
    )  
  
    if ($Age -gt 0) {  
        return "Bonjour $Nom, $Age ans"  
    }  
    return "Bonjour $Nom"  
}  
  
Get-Salutation -Nom "Alice" -Age 30
```

9.2 Paramètres avancés

```
function Set-UserLicense {  
    [CmdletBinding(SupportsShouldProcess)]  
    param(  
        [Parameter(Mandatory, ValueFromPipeline)]  
        [string]$UserId,  
  
        [Parameter(Mandatory)]  
        [string[]]$SkuId  
    )  
  
    process {  
        if ($PSCmdlet.ShouldProcess($UserId, "Attribuer licence")) {  
            # logique d'attribution  
            Write-Verbose "Licence attribuée à $UserId"  
        }  
    }  
}  
  
# -WhatIf et -Confirm (grâce à SupportsShouldProcess)  
Set-UserLicense -UserId "abc" -SkuId "sku1" -WhatIf
```

ATTRIBUT	EFFET
Mandatory	Paramètre obligatoire
ValueFromPipeline	Accepte l'entrée pipeline
ValidateSet("A","B")	Valeurs autorisées
SupportsShouldProcess	Active <code>-WhatIf</code> / <code>-Confirm</code>

9.3 Script paramétré

```
# En-tête typique d'un script .ps1
param(
    [Parameter(Mandatory)]
    [string]$CsvPath,

    [switch]$WhatIfMode
)

# Corps du script
$users = Import-Csv $CsvPath
foreach ($u in $users) {
    Write-Host "Traitement : $($u.UPN)"
}
```

10. Modules et connexion M365

10.1 Gestion des modules

```
# Installer un module (utilisateur courant)
Install-Module Microsoft.Graph -Scope CurrentUser
Install-Module ExchangeOnlineManagement -Scope CurrentUser
Install-Module MicrosoftTeams -Scope CurrentUser

# Lister les modules installés
Get-Module -ListAvailable Microsoft.Graph*

# Importer explicitement
Import-Module Microsoft.Graph.Users
```

10.2 Connexions courantes

```
# Microsoft Graph
Connect-MgGraph -Scopes "User.Read.All", "Organization.Read.All"
Get-MgContext # contexte de connexion
Disconnect-MgGraph

# Exchange Online
Connect-ExchangeOnline -UserPrincipalName admin@contoso.com
Get-Mailbox -ResultSize 10
Disconnect-ExchangeOnline -Confirm:$false

# Microsoft Teams
Connect-MicrosoftTeams
Get-Team | Select DisplayName, GroupId, Visibility
Disconnect-MicrosoftTeams
```

10.3 Exemple : lecture des licences

```
Connect-MgGraph -Scopes "Organization.Read.All"

Get-MgSubscribedSku |
    Select-Object SkuPartNumber, ConsumedUnits,
        @{ N = "Total"; E = { $_.PrepaidUnits.Enabled } } |
    Format-Table -AutoSize
```

10.4 Exemple : boucle sur un CSV

```
$import = Import-Csv .\nouveaux-users.csv

foreach ($ligne in $import) {
    $params = @{
        DisplayName      = $ligne.Nom
        UserPrincipalName = $ligne.UPN
        MailNickname      = $ligne.UPN.Split("@")[0]
        UsageLocation     = "FR"
        AccountEnabled    = $true
        PasswordProfile   = @{
            Password                        = $ligne.MotDePasse
            ForceChangePasswordNextSignIn = $true
        }
    }
    New-MgUser -BodyParameter $params
}
```

Sécurité — Ne jamais stocker de mots de passe en clair dans un script versionné. Préférer Azure Key Vault, `Get-Credential` ou des secrets d'automatisation.

11. Gestion des erreurs

```
# Arrêt sur erreur
$ErrorActionPreference = "Stop" # Stop | Continue | SilentlyContinue

# Try / Catch / Finally
try {
    Connect-MgGraph -Scopes "User.ReadWrite.All"
    New-MgUser -BodyParameter $user
}
catch {
    Write-Error "Échec : $($_.Exception.Message)"
    # journalisation, notification...
}
finally {
    Disconnect-MgGraph
}

# Erreurs non bloquantes sur une commande
Get-MgUser -UserId "inexistant" -ErrorAction SilentlyContinue

# Vérifier le succès
if (-not $?) {
    Write-Warning "La commande précédente a échoué"
}
```

11.1 Niveaux de message

```
Write-Host "Information visible" -ForegroundColor Cyan
Write-Verbose "Détail (nécessite -Verbose)"
Write-Warning "Avertissement"
Write-Error "Erreur"
# Write-Debug nécessite -Debug
```


12. Récapitulatif des instructions courantes

12.1 Aide et découverte

INSTRUCTION	DESCRIPTION
<code>Get-Help <cmdlet></code>	Aide sur une commande
<code>Get-Help <cmdlet> -Examples</code>	Exemples d'utilisation
<code>Get-Command</code>	Liste toutes les commandes
<code>Get-Command *User*</code>	Recherche par motif
<code>Get-Alias</code>	Liste des raccourcis
<code>Update-Help</code>	Met à jour l'aide locale

12.2 Fichiers et dossiers

INSTRUCTION	DESCRIPTION
<code>Get-ChildItem (ls , dir)</code>	Liste fichiers et dossiers
<code>Get-ChildItem -Recurse</code>	Parcours récursif
<code>Set-Location (cd)</code>	Change de répertoire
<code>New-Item -ItemType Directory</code>	Crée un dossier
<code>Copy-Item</code>	Copie fichier/dossier
<code>Move-Item</code>	Déplace ou renomme
<code>Remove-Item</code>	Supprime
<code>Test-Path</code>	Vérifie l'existence
<code>Get-Content (cat)</code>	Lit un fichier texte
<code>Set-Content</code>	Écrit dans un fichier
<code>Add-Content</code>	Ajoute à la fin d'un fichier

12.3 Pipeline et objets

INSTRUCTION	DESCRIPTION
<code>Where-Object (?)</code>	Filtre des objets
<code>Select-Object (select)</code>	Sélectionne des propriétés
<code>Sort-Object (sort)</code>	Trie
<code>Group-Object</code>	Regroupe par propriété
<code>Measure-Object</code>	Compte, somme, moyenne
<code>ForEach-Object (%)</code>	Traite chaque objet
<code>Format-Table</code>	Affichage en tableau
<code>Format-List</code>	Affichage en liste
<code>Out-String</code>	Convertit en chaîne
<code>Tee-Object</code>	Duplique la sortie pipeline

12.4 Données structurées

INSTRUCTION	DESCRIPTION
<code>Import-Csv</code>	Lit un fichier CSV
<code>Export-Csv</code>	Exporte en CSV
<code>ConvertFrom-Json</code>	Parse du JSON
<code>ConvertTo-Json</code>	Produit du JSON
<code>Import-Clixml</code>	Lit un XML PowerShell
<code>Export-Clixml</code>	Exporte en XML PowerShell

12.5 Variables et environnement

INSTRUCTION	DESCRIPTION
Get-Variable	Liste les variables
Set-Variable	Définit une variable
Remove-Variable	Supprime une variable
Get-Date	Date et heure courantes
Start-Sleep	Pause (secondes)

12.6 Modules et exécution

INSTRUCTION	DESCRIPTION
Install-Module	Installe un module
Update-Module	Met à jour un module
Get-Module -ListAvailable	Modules installés
Import-Module	Charge un module
Get-ExecutionPolicy	Lit la politique scripts
Set-ExecutionPolicy	Modifie la politique
Invoke-Expression	Exécute une chaîne (à utiliser avec prudence)

12.7 Microsoft Graph (admin M365)

INSTRUCTION	DESCRIPTION
<code>Connect-MgGraph</code>	Connexion à Graph
<code>Disconnect-MgGraph</code>	Déconnexion
<code>Get-MgContext</code>	Infos de session
<code>Get-MgUser</code>	Liste / lit des utilisateurs
<code>New-MgUser</code>	Crée un utilisateur
<code>Update-MgUser</code>	Modifie un utilisateur
<code>Remove-MgUser</code>	Supprime un utilisateur
<code>Get-MgGroup</code>	Liste des groupes
<code>New-MgGroup</code>	Crée un groupe
<code>Get-MgSubscribedSku</code>	Licences souscrites
<code>Get-MgGroupMember</code>	Membres d'un groupe

12.8 Exchange Online

INSTRUCTION	DESCRIPTION
<code>Connect-ExchangeOnline</code>	Connexion Exchange
<code>Disconnect-ExchangeOnline</code>	Déconnexion
<code>Get-Mailbox</code>	Liste des boîtes
<code>Get-MailboxStatistics</code>	Statistiques boîte
<code>Set-Mailbox</code>	Configure une boîte
<code>Get-TransportRule</code>	Règles de transport

12.9 Microsoft Teams

INSTRUCTION	DESCRIPTION
<code>Connect-MicrosoftTeams</code>	Connexion Teams
<code>Disconnect-MicrosoftTeams</code>	Déconnexion
<code>Get-Team</code>	Liste des équipes
<code>New-Team</code>	Crée une équipe
<code>Get-TeamUser</code>	Membres d'une équipe
<code>Add-TeamUser</code>	Ajoute un membre
<code>Get-CsTeamsMeetingPolicy</code>	Politiques de réunion
<code>Set-CsTeamsMeetingPolicy</code>	Modifie une politique

12.10 Aide-mémoire syntaxe rapide

BESOIN	SYNTAXE
Commentaire ligne	<code># ...</code>
Commentaire bloc	<code><# ... #></code>
Pipeline	<code>cmdlet1 cmdlet2</code>
Tableau	<code>@(1, 2, 3)</code>
Hashtable	<code>@{ cle = "val" }</code>
Objet anonyme	<code>[pscustomobject]{ A = 1 }</code>
Propriété calculée	<code>@{ N = "Nom"; E = { \$_.Prop } }</code>
Bloc de script	<code>{ ... }</code>
Paramètre switch	<code>[switch]\$Activer</code>
Credential sécurisé	<code>Get-Credential</code>
Chemin multi-segments	<code>Join-Path \$a \$b</code>
Tester fichier	<code>Test-Path ".\fichier.csv"</code>
Compter éléments	<code>(\$liste).Count</code>
Sortie console colorée	<code>Write-Host "OK" -ForegroundColor Green</code>

Pour aller plus loin

RESSOURCE	LIEN
Bien démarrer avec PowerShell	Microsoft Learn — PowerShell
Microsoft Graph PowerShell	Get started with Graph PowerShell
Exchange Online PowerShell	Exchange Online PowerShell V3
Teams PowerShell	Teams PowerShell overview
Référence des verbes approuvés	Approved verbs